

# Erratum to Hash Function Submission FEILIAN

FEILIAN Designer Team

October 2026

**Note.** The design of hash function FEILIAN does not change. All details should refer to the reference code that illustrates FEILIAN correctly and fully.

**Original Document Version 1.0** (Released: June 2026)

**Current Document Version 1.0.1** (Released: October 2026)

## 1 Correction to Typos in Specification

The corrections to typos in specification.pdf is summarized in Table 1.

Table 1: Correction to Specification

| Location                     | Changes                                                                                                |
|------------------------------|--------------------------------------------------------------------------------------------------------|
| Page 3, Section 2.1          | The Initial Vector:<br>$IV_7 = 0x3D84D5B5B5470917 \rightarrow IV_7 = 0x3F84D5B5B5470917;$              |
| Page 9, Section 2.2.2        | The Round Constant:<br>$(0, tw, 0, \text{Const}) \rightarrow (0, \text{Const}, 0, tw);$                |
| Page 61, Bibliography        | Bib of “Second preimage attacks on dithered hash functions”<br>[IKD+08] $\rightarrow$ [ABF+08]         |
| Page 59-68, Appendix B.2/B.3 | The printed test vectors for FEILIAN-768/512<br>wrong historical version $\rightarrow$ correct version |

## 2 Corrections to Implementation

The corrections to software and hardware implementation is summarized in Table 2.

We note that the software implementation (both reference and optimized codes) are correct. The modifications are mainly to refine the software implementation towards full-fledged version: (1) handle exceptional execution situation properly, (2) handle non-full byte string properly, and (3) correct some inconsistency in program comments.

## 3 Other modifications to Specification

We made few modification due to the community’s assessment report [1] as below:

- Page 18, Section 4.2, we provided more detailed NIST SP-800-22 test results;
- Page 19, Section 4.3.1, we unified the notations in indifferentiability proof to be fully consistent with FEILIAN, during sketching and adapting the proof of HAIFA;
- Page 37, Section 6.2, we added a footnote to further emphasize the necessity of unambiguous encoding of context and counter;
- Page 37-38, Section 6.3, we added a note to illustrated that XOF should be regarded as an variable-output-length hash function, leveraging the same compressison function FEILIAN- $f$ .

Table 2: Corrections to Software and Hardware Implementation

| Issues                                                            | Changes                                                                                                                                                                                                                                                                                |
|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Software Implementation</b>                                    |                                                                                                                                                                                                                                                                                        |
| Allocation Failure                                                | pad_message() returns 0 if allocation fails.<br>The internal hash function detects this and returns -1;<br>CryptHash() propagates the error.                                                                                                                                           |
| Non-full-byte string                                              | After copying the input, pad_message() masks unused low-order bits in the final byte when <i>msg_bits</i> % 8 != 0.<br>Those unused bits are cleared before compression.                                                                                                               |
| Length Representability                                           | Before converting the input length to size_t,<br>CryptHash() checks whether it is representable.<br>pad_message() calculates rounded-up byte and block counts using division and remainder,<br>then checks num_blocks > SIZE_MAX / BLOCK_BYTES before calculating the allocation size. |
| Non-compliant usage                                               | check whether the requested digest length matches the current version(512/768/1024) and returns an error if it does not.                                                                                                                                                               |
| Counter-word comments                                             | hash_bits[0] holds the high word;<br>hash_bits[1] holds the low word.                                                                                                                                                                                                                  |
| <b>Hardware Implementation</b>                                    |                                                                                                                                                                                                                                                                                        |
| current block length<br>omitted in 1SC/4SC/8SC RTL                | revised the implementation of 1SC/4SC/8SC core to include the current block length in the compression input                                                                                                                                                                            |
| Block/final flag/cumulative length<br>be transaction-consistent   | latch the message block, tweak<br>(including the final flag and cumulative bit count)<br>and initial chaining state,<br>when a compression transaction is accepted,<br>preserving them throughout compression                                                                          |
| Counter-word                                                      | All RTL implementations use the same counter-word ordering:<br>domain word 2 contains the upper 64 bits;<br>and domain word 3 contains the lower 64 bits.                                                                                                                              |
| digest variants;<br>parameter propagation                         | all four architectures support 512/768/1024 configurations.<br>The top-level DIGEST_BITS parameter is forwarded to the core.<br>It derives the version value and output-word count.<br>Digest selection and MMIO address bounds now follow the configured output length.               |
| Intermediate state exposure;<br>clearing requirements for key-use | The 1SC/2SC/4SC cores restrict the digest output to completed hashes<br>The top-level MMIO read path is also restricted to completed hashes                                                                                                                                            |
| Readme typo                                                       | fixed the README to indicate:<br>the four RTL architectures support three instances;<br>the architecture names indicate SubColumn parallelism.                                                                                                                                         |

## References

- [1] Mounir Idrassi. FEILIAN: technical assessment of the submitted specification and implementations, 2026.